

ESP32 Wetter Station Teil 2

Anthony Timmel

12. Mai 2026

Inhaltsverzeichnis

I	Einführung	2
1	Grundlagen	4
1.1	Pub/Sub Messaging	4
1.2	MQTT-Client	5
II	MQTT-Client im ESP32	6
2	Internetverbindung für den ESP32	7

Teil I

Einführung

Im Rahmen des ersten Teils dieser Dokumentation setzen wir uns intensiv mit der initialen Aufgabenstellung aus dem Unterricht des Lernfeldes 7 (LF7) auseinander. Auf den nachfolgenden Seiten wird Ihnen das Projekt *Wetterstation ESP32* in seiner zweiten Phase detailliert und in schriftlicher Form präsentiert, um einen fundierten Überblick über die Zielsetzung und den Aufbau zu geben.

Diese Dokumentation ist der zweite Teil der Dokumentationsreihe, über den *ESP32* als Wetterstation.

Kapitel 1

Grundlagen

Innerhalb des zweiten Teils, kommen neue softwarebasierte Technologien zum Einsatz. Damit, bei der Referenzierung auf diese Begriffe & bei dem Ablauf der Beschreibung des Programmcodes keine Komplikationen in dem Verständniss des Ablaufs auftreten, werden die Begrifflichkeiten, im Umfang der zweiten Aufgabenstellung erklärt.

1.1 Pub/Sub Messaging

Pub/Sub Messaging ist eine Protokollfamilie. Hierbei werden Nachrichten in sogenannten **Channels** oder **Topics** versendet. Man könnte dies mit den verschiedenen Bereichen in einem Amt vergleichen, denn je nach Channel, geht die Nachricht an andere Empfänger über. Jedoch bleiben für den Clienten die Empfänger unbekannt.

Pub/Sub ist ein serverbasierter Nachrichtenprotokoll, d.h. die Nachricht, die ein Client versendet, werden an einen Server gesendet. Dieser *Verwaltet* die Eingehenden & Ausgehenden Verbindungen, den andere Clients können sich an diese Topics anlinken. Dadurch wird die Nachricht, sobald diese bei dem Server ankommt, automatisch an diese Clients versendet. In dieser Grafik sieht man zwei verschiedene **Publisher**

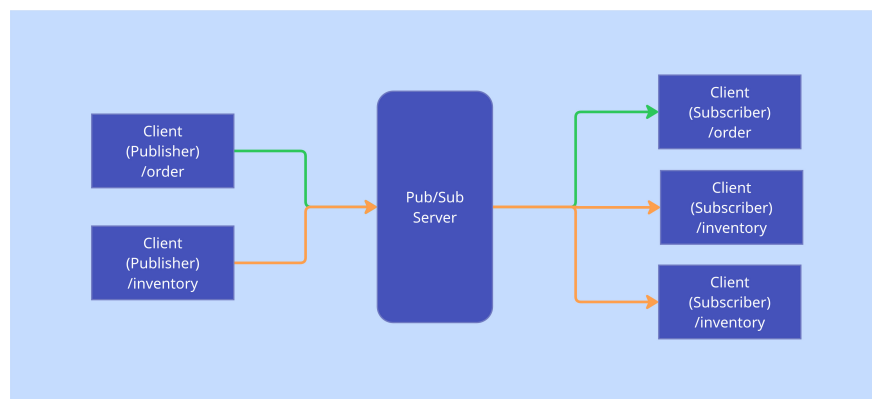


Abbildung 1.1: Grafische Darstellung des Pub/Sub Verfahrens

Clients. Diese senden eine Nachricht auf **/order** & **/inventory**. Beide senden die Nachricht mit dem Channel/Topic an den Pub/Sub Server. Hinter dem Pub/Sub Server befinden sich drei unterschiedliche Clients. Zwei dieser Clients hören auf Nachrichten innerhalb des **/order** Channels & einer auf den **/inventory**. Die Verbindung zwischen Client & Server bestehen aus WebSocket Verbindungen, damit der Server & der Client sich gegenseitig Nachrichten zusenden zu können.

1.2 MQTT-Client

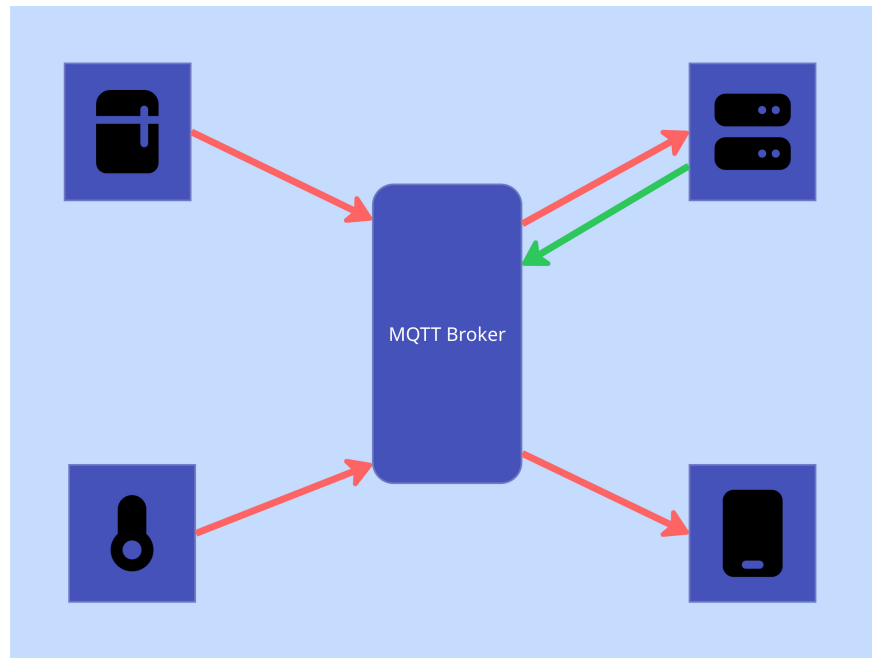


Abbildung 1.2: Grafik von MQTT

Diese Grafik verbildlicht die Kommunikation zwischen den einzelnen Geräten wie den Kühlschrank, Thermometer, Server & einem Handy. Hierbei werden die Daten mithilfe von dem MQTT-Client an den Broker gesendet. Der Broker sendet diese Daten dann an alle Clients, die den **Channel** subscribed haben.

Innerhalb dieses zweiten Teils werden Daten an einen externen Dienst versendet. Dafür gibt es verschiedene Wege, diesen Prozess zu absolvieren.

Das MQTT-Protokoll ist ein klassisches Pub/Sub Modell in der IoT Welt. Hierbei agiert es als leichtgewichtiges Messaging Protokoll für die Übermittlung von Daten, von unseren Client an den Server.

Teil II

MQTT-Client im ESP32

Kapitel 2

Internetverbindung für den ESP32

Damit Daten überhaupt von dem ESP32 an den MQTT-Broker übermittelt werden können, benötigt der ESP32 eine Internetverbindung.

Für die technische Umsetzung wurde sich auf die [ESP32 WLAN Modul Dokumentation](#) bezogen.

```
1  import network
2
3  wlan = network.WLAN(network.STA_IF)
4
5  def do_connect():
6      wlan.active(True)
7      if not wlan.isconnected():
8          print('connecting to network...')
9          wlan.connect('FI24-Hotspot', 'BszWsw11#')
10         while not wlan.isconnected():
11             print("WLAN is not connected")
12             pass
13
14     print('network config:', wlan.ifconfig())
15
16 do_connect()
17
```

Zuerst importieren wir das Modul **network**. In Zeile 3 deklarieren wir das WLAN-Modul des ESP32, indem wir aus dem Netzwerk-Modul die Klasse **WLAN** instanziiieren. **network.STA_IF** bedeutet hierbei, das wir es als Station Interface deklarieren, also ein stationärer Zugriffspunkt (Access Point).

Damit wir später im Programmcode einfacher eine W-LAN Verbindung herstellen können, deklarieren wir die Funktion **do_connect()**. Innerhalb des Funktionsbodies, aktivieren wir zuerst die WLAN-Funktionalität, damit wir uns später mit einem Access Point verbinden können. Wir überprüfen zuerst, ob wir mit einem Access-Point verbunden sind. Dies verhindert, dass wir die Verbindung zu dem Access-Point neu aufbauen, obwohl wir bereits eine Verbindung haben.

In Zeile 9 verbinden wir uns mit dem definierten Access Point & überprüfen in der while-Schleife dann so lange, ob die Verbindung erfolgreich war. Erst nach einer erfolgreichen Verbindung lassen wir die while-Schleife los & geben die IP-Informationen in die Konsole aus.

Damit die WLAN Verbindung initialisiert werden kann, wird die Funktion **do_connect()** aufgerufen.